Final Project Report

Jeremy Carleton

May 2024

1 Introduction to the Main Results

Altschuler and Parrilo's paper [1] shows for the first time that an acceleration in terms of the iteration complexity of gradient descent for general smooth and strongly-convex functions can be achieved simply by choosing the correct dynamic stepsize schedule, known as the Silver stepsize schedule. In this work, we summarize Altschuler's main contributions and analyze how the Silver stepsize schedule generalizes to the constrained minimization setting. The primary result reported by Altschuler is that, while constant stepsizes only guarantee an iteration complexity that is $\Theta\left(\kappa \log(\frac{1}{\epsilon})\right)$, the Silver stepsize schedule improves this to $\Theta\left(\kappa^{\log_\rho 2} \log(\frac{1}{\epsilon})\right)$ where $\rho = 1+\sqrt{2}$. The improvement is thus by a factor of $\frac{\kappa^{\log_\rho 2}}{\kappa} \approx \kappa^{0.7864-1} = \kappa^{-0.2136}$.

The class of functions considered in [1] are those which are smooth and strongly convex; the sequel [2] drops the strong convexity assumption. It should be noted that the analysis used to prove the acceleration achieved by the Silver stepsize schedule can only bring improvement for the class of smooth convex functions, so the strong convexity assumption is the only one which could be improved upon in the first place. In the smooth and strongly convex case, Altschuler quantifies the relative sharpness of a function's gradient using its condition number $\kappa = \frac{M}{m}$.

Altschuler notes that previous methods used to analyze the convergence of gradient descent are quite short-sighted because they consider the improvement that each step makes in isolation, where each step is subject to the worst-case function over the entire function class. This analysis fails to exploit the fact that these worst-case functions need not be aligned with each other such that there is a single function in the function class that can provide such poor behavior. A useful way to evaluate the effectiveness of a stepsize is to analyze the worst-case convergence rate (1), which tells how much closer the algorithm gets to the optimal point by taking a gradient

descent step of that size.

$$R(t_k, \mathcal{F}) = \sup_{f \in \mathcal{F}, x_k \neq x^*} \frac{||x_k - t_k \nabla f(x_k) - x^*||_2}{||x_k - x^*||_2} \tag{1}$$

To analyze the Silver stepsize schedule, Altschuler takes a global approach to analyzing the progress made in each step of gradient descent. This takes the form of hedging between stepsizes which are more reasonable in regions where the function is steep and stepsizes which are more reasonable in regions where the function is more flat. This is illustrated using the case of quadratic functions, where it has long been known that pairing long and short stepsizes together is superior to keeping the stepsize constant. Intuitively, although the stepsize which maximizes (1) for any individual step in isolation is given by $\frac{2}{M+m}$, it is feasible to analyze the worst-case convergence rate over multiple steps, which gives very different results. The optimal step sizes for two steps are the inverses of the roots of the Chebyshev polynomial of the first kind obtained by minimizing the L_∞ norm of $p(H) = (1 - \alpha H)(1 - \beta H)$ over the interval $[m, M]$ where H is the hessian matrix of the quadratic function f. This discrepancy arises because when optimizing the individual steps in isolation, one must use (2) as the convergence rate for both of the stepsizes, meaning that the range of possible values for the worst-case eigenvalue λ on one step is not affected by the value of λ used on the other step.

$$R(t_k, \mathcal{F}) = \sup_{m \leq \lambda \leq M} |1 - t_k \lambda| \tag{2}$$

In reality, however, the function over which one is optimizing does not change between steps, so λ can be constrained such that it takes the same value in both steps. This shows that the conventional analysis of the convergence rate is overly pessimistic for non-constant stepsize choices, since there is likely to be a difference in the worst-case function for two distinct stepsizes. With the Silver stepsize schedule, Altschuler shows that this pessimism can be fixed for the more general function class of smooth and strongly convex (or just smooth and convex) functions.

To derive the Silver stepsize schedule, Altschuler begins with the two-step case and uses it to build a stepsize schedule for $n = 2^m$ for any positive integer m. The two-step schedule is the solution to the following system of equations:

$$(M\alpha - 1)(M\beta - 1) = (1 - \alpha m)(1 - \beta m) = \frac{(1 - \alpha m)(M\beta - 1)}{1 + \alpha(M - m)} \tag{3}$$

An important difference between the solution to (3) and the minimizer of the L_∞ norm of $p(H) = (1 - \alpha H)(1 - \beta H)$ used in the quadratic case is that α^* and β^* are not interchangeable. In particular, the smaller stepsize must be taken first. The solution to (3) can be expressed as in (4), and the worst-case convergence rate achieved is given in (5):

$$\alpha^* = \frac{2}{m+S}, \beta^* = \frac{2}{2M+m-S} \tag{4}$$

$$R^* = \frac{S-M}{2m+S-M} \tag{5}$$

where $S = \sqrt{M^2 + (M-m)^2}$. Altschuler notes that comparing (5) with the optimal two-step convergence rate for a constant stepsize,

$$\left(\frac{M-m}{M+m}\right)^2$$

indicates that R^* is strictly smaller, and hence the Silver stepsize schedule will provably make more progress over the two steps than a constant stepsize in the worst case.

2 Upper Bound on the Convergence Rate

To prove the upper bound on the convergence rate of the Silver stepsize schedule, Altschuler exploits the fact that the sequence of function values and gradients observed when running gradient descent must all be consistent with the actual identity of f, and hence they can be used to restrict the set of functions which f must be in. This is similar to exploiting the fact that λ must be consistent between steps in the quadratic case. Using known conditions for $\mathcal{F}$-interpolability, Altschuler derives the following condition for the claimed convergence rate to be true:

$$R^2||x_0 - x^*||_2^2 - ||x_2 - x^*||_2^2 = \sum_{i \neq j \in \{0,1,*\}} \lambda_{ij} Q_{ij} \tag{6}$$

where

$$\begin{aligned} Q_{ij} = \quad & 2(M-m)(f(x_i) - f(x_j)) + 2\langle M\nabla f(x_j) - m\nabla f(x_i), x_j - x_i\rangle \\ & - ||\nabla f(x_i) - \nabla f(x_j)||_2^2 - Mm||x_i - x_j||_2^2 \end{aligned}$$

and the index $*$ corresponds to x^*. The proof for the two-stepsize case is then completed by finding appropriate non-negative multipliers λ_{ij}. Explicit expressions for these multiplier are given in equation 2.9 of [1].

3 Lower Bound on the Convergence Rate

The proof of the lower bound on the convergence rate for general smooth and strongly convex functions, which also proves the optimality of the Silver stepsize ratio, requires constructing a set of functions that covers the worst case for any given stepsize schedule. Altschuler notes that it is sufficient to consider only stepsizes in the range $[\frac{1}{M}, \frac{1}{m}]$ since it can be shown that any time a stepsize outside that range is used, the worst-case convergence rate over the functions considered can be improved by clipping it to that range. This proof is done for the case of one-dimensional functions, and the set of "bad" functions consists of two quadratics with the two extreme values of $f''(x)$ and two piecewise quadratics which transition between those values. The first piecewise quadratic transitions at $x = 0$ while the second transitions at $x = \frac{1-m\alpha}{1+\alpha(M-m)}$; this asymmetry in the fourth function is the reason why the Silver stepsize schedule has a unique order. The significance of the piecewise functions is that they capture the added difficulty introduced by considering the set of all smooth and strongly convex functions rather than just the quadratics. For a general convex function it is possible for the curvature on one side of the optimal point to be very different from that of the other side, and that can lead to cases where the algorithm overshoots and becomes trapped in a flat region. For an algorithm using non-adaptive stepsizes, this can present a serious challenge.

4 Properties of the Silver Stepsize Schedule

For horizons that are powers of two, the Silver stepsize schedule is first constructed in a normalized form such that all stepsizes are in the range $(0, 1]$, and then rescaled using a linear fractional transformation. The construction of the normalized schedule begins with constructing two sequences y_n and z_n using the following rules:

$$y_1, z_1 = \frac{1}{\kappa}, y_n z_n = z_{n/2}^2 \text{ and } z_n - y_n = 2(z_{n/2} - z_{n/2}^2)$$

Lemma 3.1 of [1] notes that $y_n \leq z_n$, with the inequality being strict for $\kappa = \frac{1}{m} > 1$ (in this section of the paper, the function is assumed to be

rescaled such that $M = 1$). Hence, in the two-step case explained earlier, y_n is analogous to the smaller stepsize which comes first and z_n is analogous to the larger stepsize which comes second. In the general case, z_n corresponds to the final stepsize in a sequence of n steps when n is any power of two greater than one. The lemma also mentions that z_n increases monotonically to a limit of 1. As illustrated in figure 5 of [1], z_n converges to 1 faster when κ is smaller.

The actual stepsizes used in the Silver stepsize schedule are created by applying the transformation ψ defined in equation (3.5) of [1] to the sequences y_n and z_n and then arranging them in a fractal pattern. As in equation (3.4) of [1], we let $a_n = \psi(y_n)$ and $b_n = \psi(z_n)$. The construction of the Silver stepsize schedule takes place recursively, starting from the $n = 1$ case where the stepsize is $b_1 = \psi(\frac{1}{\kappa})$. For the $n = 2$ case, this is split into the two stepsizes $a_2 = \psi(y_2)$ and $b_2 = \psi(z_2)$. Given the Silver stepsize schedule for n steps (where n is any power of two), the schedule for $2n$ steps is obtained by copying the first $n - 1$ steps exactly, adding a_n as the nth step, copying the first $n - 1$ steps again, and adding b_n as the $2n$th step. Thus the first $2n - 1$ steps oscillate in a periodic fashion, and the $2n$th step is larger than all the steps before it.

The convergence rate of the Silver stepsize schedule (or simply the Silver convergence rate) is given in terms of z_n in equation (3.9) of [1]. Since z_n monotonically increases to 1, it follows that the Silver convergence rate monotonically decreases, and furthermore it is 0 at the limit as $n \to \infty$. In theorem 4.1 of [1], Altschuler shows that for horizons up to a certain amount n^*, the Silver convergence rate improves with the horizon at a super-exponential rate; but after that point, the improvement is only exponential. These two regimes are called the acceleration regime and the saturation regime. The saturation regime emerges because the difference between a_n and b_n becomes smaller and smaller as n increases, and eventually the difference between the first $\frac{n}{2}$ and the second $\frac{n}{2}$ stepsizes becomes negligible (since they only differ in the last step). This indicates that when using the multi-step descent analysis technique, optimizing over a larger number of consecutive steps is only useful up to a certain point; eventually, it becomes roughly as good to just repeat the previous sequence of steps exactly.

5 Generalizing to Constrained Minimization

5.1 Projected Gradient Descent on Quadratic Functions

To investigate the performance of the Silver stepsize schedule in constrained minimization, we first consider the quadratic case analyzed in section 2.1 of [1]. In particular, $f(x) = \frac{1}{2}x^T Hx$ where $m\mathbf{I} \preceq H \preceq M\mathbf{I}$. In this case, as noted previously, in the unconstrained setting the optimal choice of step size for $n = 2$ is given by the roots of the corresponding Chebyshev polynomial (denoted by α^* and β^*). As noted in [1], this results in one longer step and one shorter step. Now, we will suppose there is a convex set $\mathcal{X}$ such that $x_0 \in \mathcal{X}$, and our goal is to solve the following optimization problem:

$$\min_x f(x) \text{ subj. to } x \in \mathcal{X}$$

In situations where $0 \in \mathcal{X}$ and the distance from 0 to the boundary of the constraint set is larger than $t_0||\nabla f(x_0)||_2$, one would not expect the constraint set to have an impact on the trajectory followed by gradient descent on a quadratic function. This is because the first step can either undershoot, in which case x_1 will be in a region with a lower gradient than before (and hence the next step is likely to be smaller), or it can overshoot but fall short of the boundary, in which case $-\nabla f(x_1)$ will be pointing away from the boundary and towards 0. In either case, the following step will not be able to reach the boundary of the constraint set unless the step size is significantly larger. However, when the point 0 lies close enough to the boundary of the constraint set or $0 \notin \mathcal{X}$, the algorithm is expected to outside of the constraint set, resulting in a projection. This projection will interfere with the optimality of the recommended stepsize schedule since $P_{\mathcal{X}}(x_k - t_k\nabla f(x_k))$ may be far from $x_k - t_k\nabla f(x_k)$. However, by the non-expansiveness of the projection operator, we know that $P_{\mathcal{X}}(x_k - t_k\nabla f(x_k))$ will not be any further from 0 than $x_k - t_k\nabla f(x_k)$ was, and in fact it may be much closer. This suggests that the cost of overshooting can be much lower in cases where the optimal point is close to the boundary of $\mathcal{X}$ than it otherwise would be.

To illustrate this, consider the 2D case and suppose we are given the bounds $m = 1$ and $M = 3$ but $H = 3\mathbf{I}$ (i.e. the function is as sharp as we can allow). The Chebyshev schedule for $n = 2$ gives the step sizes

$$\alpha = \left(2 - \frac{1}{\sqrt{2}}\right)^{-1} \approx 0.77, \beta = \left(2 + \frac{1}{\sqrt{2}}\right)^{-1} \approx 0.37$$

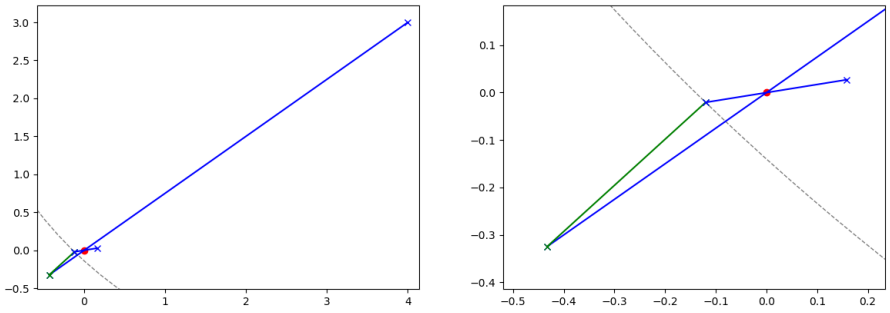


Figure 1: The trajectory followed by projected gradient descent when the smaller step size from the Chebyshev schedule is used first. Blue lines represent gradient descent steps, green lines represent projection steps, and the gray dashed curve is the boundary of the constraint set. The left image shows the full trajectory, and the right image is zoomed in to show the details of the projection and the second step.

We also let x_0 be $(4, 3)^T$. In the unconstrained case, it does not matter which order we use these step sizes in; in both cases, the value of $f(x_2)$ will be 0.7653. However, suppose that we let $\mathcal{X}$ be a ball of radius 5 centered at $(\frac{4.9}{\sqrt{2}}, \frac{4.9}{\sqrt{2}})^T$, in which case the distance between 0 and the boundary of $\mathcal{X}$ is 0.1. The trajectory that we get when running projected gradient descent for two steps with the smaller step size β used first is shown in Figure 1. We can see that due to the sharpness of f, the first step using the smaller step size overshoots 0 and also moves outside of the constraint set. The projection makes up for most of this overshoot, bringing x_1 quite close to 0. However, because of this the second step overshoots as well, putting x_2 further from 0 than x_1 was. In the end, we get $f(x_2) = 0.03884$.

On the other hand, the trajectory that we get when we use the larger step size α first is shown in Figure 2. As expected, the overshoot on the first step is significantly worse than in the previous case; in fact, it is roughly 10 times worse. However, due to the projection, x_1 is not significantly further away from 0 than it was in the previous case. The second step using the smaller step size is able to get very close to 0; indeed, we get $f(x_2) = 0.00303$, a whole order of magnitude smaller than in the previous case. These results both confirm that the use of projection onto a constraint set can mitigate the issue of overshoot and suggest that when hedging is used, it is better to lead with the larger step size if the step is likely to move outside of the constraint set.

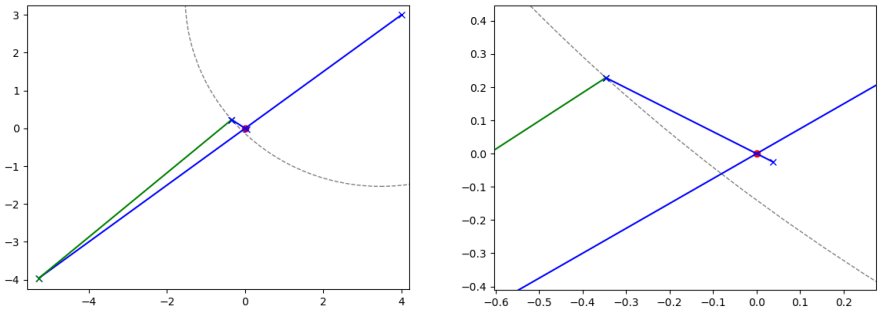


Figure 2: The trajectory followed by projected gradient descent when the larger step size from the Chebyshev schedule is used first. Blue lines represent gradient descent steps, green lines represent projection steps, and the gray dashed curve is the boundary of the constraint set. The left image shows the full trajectory, and the right image is zoomed in to show the details of the second step.

Since the Silver stepsize schedule always takes the shorter step first, it may not be so advantageous in cases where it is common for the algorithm to move outside of the constraint set. However, once this is no longer the case, the Silver stepsize will work as well as in the unconstrained case since there will not be any more projections taking place. One way to adapt the Silver stepsize schedule to the constrained minimization setting could be to only modify the step size for steps which take place immediately following a projection. For example, one could discount the size of the next step following a projection based on how far the iterate x_k was moved by the projection. Another way could be to "reset" the stepsize schedule each time a projection takes place. Note that either of these modifications would make the stepsizes adaptive, whereas the original Silver stepsize schedule is not.

To observe the impact of the constraint set on the convergence of the Silver stepsize schedule, we ran numerical simulations where both gradient descent (GD) and projected gradient descent (PGD) are performed on a quadratic function. The setting was chosen to be similar to the example given at the beginning of this section, except that the center of the constraint set and the starting point for the algorithms were chosen randomly. We ensured that the boundary of the constraint set was a distance of 0.1 from the optimal point of 0, and that the starting point was a distance of 1 away from the center of the constraint set. The condition number κ was set to 4, which was a loose bound since $||\nabla^2 f||_2 = 1$. When using the Silver

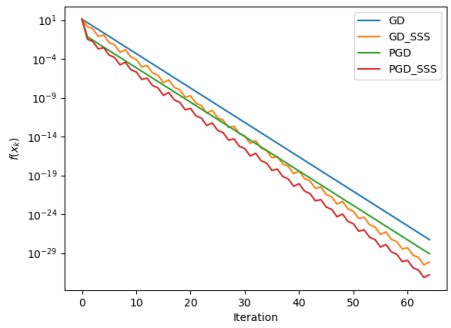


Figure 3: Evolution of function value over time for gradient descent in the unconstrained case (GD) and the constrained case (PGD), both using constant stepsizes and the Silver stepsize schedule (SSS)

stepsize schedule, the schedule was constructed for each power of two in the binary expansion of the horizon, and these schedules were concatenated together going from longest to shortest. An example of the trajectories for GD and PGD with and without the Silver stepsize schedule (SSS) is provided in Figure 3. I found that across different seeds for the random number generator, the relative performance of the algorithms remained the same.

In Figure 3, we see that contrary to the intuition gained from performing experiments in the two-step case, the Silver stepsize schedule does not seem to get less benefit out of the projection even though it takes the shorter step first. The trajectories for the algorithms using the Silver stepsize schedule converge at a faster rate (as indicated by their greater slope), and the gap between them is similar to the gap seen for the constant stepsize algorithms. This gap is entirely due to the projection which takes place in the constrained case.

5.2 Projected Gradient Descent on Non-Quadratic Functions

We will now observe how the Silver Stepsize Schedule performs in a more challenging constrained optimization setting where the second derivative varies with x, and test the two methods for helping it respond to projection. For this purpose, we consider the following 1-dimensional function restricted

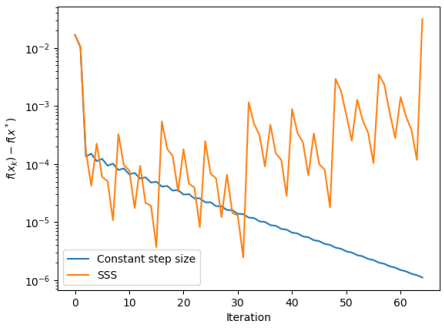


Figure 4: Evolution of suboptimality over time for projected gradient descent on the problem given in (7), both using constant stepsizes and the Silver stepsize schedule (SSS)

to the positive real numbers:

$$f(x) = C\left(\frac{1}{x} + \gamma x\right)$$

where C is a scaling factor used to ensure that $M = 1$ and γ is used to change the curvature of the function. We impose a lower bound $L > 0$ and an upper bound U on the values which x may take, so the overall problem is

$$\min_x f(x) \text{ subj. to } L \leq x \leq U \tag{7}$$

In the following experiment, the bounds were $L = 0.5$ and $U = 1.5$, and the condition number was $\kappa = 54$ (we ensured $M = 1$ and $m = \frac{1}{54}$ by setting $C = \frac{1}{16}$). The starting point was set to $x_0 = 0.6$. In the following plots, the y-axis measures the difference between the function value at the current iterate and the optimal function value (which unlike in the quadratic case is nonzero). Figure 4 shows the performance of projected gradient descent using a constant stepsize and using the Silver stepsize schedule. Both algorithms see a major improvement in suboptimality during the first two iterations due to the projection, but the Silver stepsize schedule becomes unstable afterwards and begins to diverge while the constant stepsize shows continuous improvement with linear convergence.

Since the projection appears to be the cause of the divergence in the SSS algorithm, we consider two potential ways to mitigate the issue. The first is to multiply the next stepsize by a compensation factor each time a

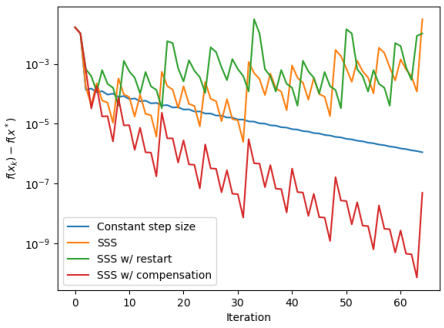


Figure 5: Evolution of suboptimality over time for projected gradient descent on the problem given in (7), using the algorithms from Figure 4 along with the two modifications to the SSS

projection occurs; this compensation factor is calculated according to (8). The compensation factor is the ratio between $||x_{k+1} - x_k||_2$ and what that distance would have been without the projection.

$$c = \begin{cases} |\frac{x_k - L}{t_k \nabla f(x_k)}| & x_k - t_k \nabla f(x_k) < L \\ |\frac{U - x_k}{t_k \nabla f(x_k)}| & x_k - t_k \nabla f(x_k) > U \end{cases} \tag{8}$$

The second way is to restart the stepsize schedule each time a projection occurs by generating the Silver stepsize schedule for a horizon equal to the remaining number of iterations and switching to that stepsize schedule. Figure 5 shows the performance of the modified algorithms under the same conditions as in Figure 4. Clearly, while the method of restarting does not perform any better than the original SSS algorithm, introducing the compensation factor prevents divergence and actually allows for faster convergence than the constant stepsize algorithm. This shows that there are cases where such an adjustment can help adapt the Silver stepsize schedule to the constrained minimization setting. Of course, there may also be cases where the adjustment actually harms the convergence rate.

References

[1] J. M. Altschuler and P. A. Parrilo. Acceleration by stepsize hedging i: Multi-step descent and the silver stepsize schedule, 2023.

[2] J. M. Altschuler and P. A. Parrilo. Acceleration by stepsize hedging ii: Silver stepsize schedule for smooth convex optimization, 2023.